



Chapter 4 : Intermediate SQL

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Outline

- Join Expressions
- Views
- Transactions
- Integrity Constraints
- SQL Data Types and Schemas
- Index Definition in SQL
- Authorization



Cartesian product

- Cartesian product combines **every tuple of one relation with every tuple of another relation, without any condition.**

In Relational Algebra

$R \times S$

R times S

If

R has **m** tuples

S has **n** tuples

→ Result has **$m \times n$ tuples**

Cartesian Product ($R \times S$)

Combines every tuple of R with every tuple of S.

No condition. No checking. Just pairing.

If 1000 students and 500 courses → how many rows?

STUDENT $\times$ TAKES

→ Every student is paired with **every course**, meaningful or not.

Cartesian product creates all combinations, not correct relationships



Cartesian product: STUDENT × COURSE

STUDENT		COURSE	
SID	Name	CID	Course
S1	Ravi	C1	DBMS
S2	Asha	C2	AI
		C3	ML
SID	Name	CID	Course
S1	Ravi	C1	DBMS
S1	Ravi	C2	AI
S1	Ravi	C3	ML
S2	Asha	C1	DBMS
S2	Asha	C2	AI
S2	Asha	C3	ML



Joined Relations

- **Join operations** take two relations and return as a result another relation.
- A join operation is a **Cartesian product** which requires that tuples in the two relations match (under some condition).
- It also specifies the attributes that are present in the result of the join
- The **join operations** are typically used as subquery expressions in the from clause
- **Three types of joins:**
 - **Natural join**
 - **Inner join**
 - **Outer join**



Natural Join in SQL

- Natural join is the cartesian product of two relations that matches tuples with the same values for all common attributes
- It automatically matches related rows → **NATURAL JOIN**.
- It retains only one copy of each common column.
- Example:
 - *student natural join takes*
- In relational algebra
$$student \bowtie takes$$

Natural join automatically
uses this common column



Natural Join in SQL

STUDENT		TAKES	
SID	Name	SID	CID
S1	Ravi	S1	C1
S2	Asha	S2	C2
S2	Asha	S2	C3

SID	Name	CID
S1	Ravi	C1
S2	Asha	C2
S2	Asha	C3

Natural Join = Cartesian Product + Auto condition + Remove duplicate column



Student Relation

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>tot_cred</i>
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120



Takes Relation

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>grade</i>
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A
12345	CS-347	1	Fall	2017	A
19991	HIS-351	1	Spring	2018	B
23121	FIN-201	1	Spring	2018	C+
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	B+
45678	CS-319	1	Spring	2018	B
54321	CS-101	1	Fall	2017	A-
54321	CS-190	2	Spring	2017	B+
55739	MU-199	1	Spring	2018	A-
76543	CS-101	1	Fall	2017	A
76543	CS-319	2	Spring	2018	A
76653	EE-181	1	Spring	2017	C
98765	CS-101	1	Fall	2017	C-
98765	CS-315	1	Spring	2018	B
98988	BIO-101	1	Summer	2017	A
98988	BIO-301	1	Summer	2018	<i>null</i>



student natural join takes

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>tot_cred</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>grade</i>
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2017	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2017	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2017	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2017	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2018	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2017	A
19991	Brandt	History	80	HIS-351	1	Spring	2018	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2018	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2017	B-
45678	Levy	Physics	46	CS-101	1	Fall	2017	F
45678	Levy	Physics	46	CS-101	1	Spring	2018	B+
45678	Levy	Physics	46	CS-319	1	Spring	2018	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2017	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2017	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2018	A-
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2017	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2018	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2017	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2017	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2018	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2017	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2018	<i>null</i>



Natural Join Use Example

- A JOIN clause is used to combine rows from two or more tables, based on a related column between them.
- Let's look at a selection from the "Orders" table:
- **List the names of instructors along with the course ID of the courses that they taught**
 - **SELECT name, course_id**
 - **FROM instructor NATURAL JOIN takes;**
- The same query without the use of natural join, **with a where clause?**
 - **SELECT name, course_id**
 - **FROM instructor, takes**
 - **Cartesian Product**
 - **WHERE instructor.ID = takes.ID**
 - **Removes wrong combinations**
 - This is called an **implicit join** or **condition join**



Natural Join Structure in SQL

- SELECT A1, A2, ... A_n
- FROM r1 NATURAL JOIN r2 NATURAL JOIN r3
- WHERE P;
- The **from** clause can have multiple relations combined using natural join:

```
select A1, A2, ... An
from r1 natural join r2 natural join .. natural join rn
where P;
```

- Result of (r1 NATURAL JOIN r2) is then NATURAL JOINed with r3, and so on.
- SELECT Name, Title
- FROM student NATURAL JOIN enroll NATURAL JOIN course;



Dangerous in Natural Join

- **Beware of unrelated attributes with same name which get equated incorrectly**
- Example -- **List the names of students along with the titles of courses that they have taken**

- Correct version

```
select name, title
from student natural join takes, course
where takes.course_id = course.course_id;
```

Result table columns: **SID, Name, Dept, Course_ID**

Course table columns: **Course_ID, Title, Dept**

This gives **all courses students actually took**, even if department differs.

- Incorrect version

```
select name, title
from student natural join takes natural join course;
```

If Ravi (CSE) takes an ECE course, result disappears

- After every natural join, check what columns remain — they may join again later without you knowing. → ALL same-name columns.
- This query omits all (student name, course title) pairs where the **student takes a course in a department other than the student's own department**.
- The correct version (above), correctly outputs such pairs.



Inner Joins

- ❑ INNER JOIN is used to combine rows from two or more tables based on a related column.
- ❑ It returns only the rows that have matching values in both tables, filtering out non-matching records.
- ❑ Combine rows from multiple tables by specifying matching criteria
 - ❑ Usually based on primary key – foreign key relationships
- ❑ Return only rows where a match is found in both input tables
- ❑ Match rows based on attributes supplied in predicate
- ❑ **If join predicate operator is =, also known as equi-join**



Join Condition

- The **on** condition allows a general predicate over the relations being joined
- This predicate is written like a **where** clause predicate except for the use of the keyword **on**
- **SELECT columns FROM table1
(INNER)JOIN table2
ON table1.column_name = table2.column_name;**

- Query example:

- Query

```
select *  
from student join takes on student_ID = takes_ID
```

- The **on** condition above specifies that a tuple **from student** matches a tuple **from takes** if their **ID** values are equal.

- Equivalent to:

```
select *  
from student , takes  
where student_ID = takes_ID
```



Outer Join

An **Outer Join** is a join operation that includes rows that have a match, plus rows that do not have a match in the other table.



Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and **then adds tuples from one relation that does not match tuples in the other relation to the result of the join.**
- Uses *null* values.
- Three forms of outer join:
 - **left outer join**
 - **right outer join**
 - **full outer join**



Outer Joins

- **Left outer join:** all rows from the table on the left (listed first in the query) will be included; matching rows only from the table on the right will be included
- **Right outer join:** all rows from the table on the right will be included; matching rows only from the table on the left will be included
- **Full outer join:** all rows from both tables will be included regardless of matches



Example

EmployeeID	Name	DepartmentID
1	John	101
2	Sarah	102
3	Michael	NULL
4	Emma	103

DepartmentID	DepartmentName
101	HR
102	IT
103	Marketing
104	Finance



LEFT OUTER JOIN

- **LEFT OUTER JOIN (or LEFT JOIN)** returns all rows from the left table and the matching rows from the right table.
- If there is no match, the result will include NULL values for columns from the right table.
- **SELECT Employees.Name, Employees.DepartmentID, Departments.DepartmentName FROM Employees LEFT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID**

Name	DepartmentID	DepartmentName
John	101	HR
Sarah	102	IT
Michael	NULL	NULL
Emma	103	Marketing



RIGHT OUTER JOIN

- **RIGHT OUTER JOIN (RIGHT JOIN)**
returns all rows from the right table and the matching rows from the left table.
- If there is no match, the result will include NULL values for columns from the left table.

- **SELECT Employees.Name,
Employees.DepartmentID,
Departments.DepartmentName
FROM Employees
RIGHT JOIN Departments
ON Employees.DepartmentID =
Departments.DepartmentID;**

Name	DepartmentID	DepartmentName
John	101	HR
Sarah	102	IT
Emma	103	Marketing
NULL	NULL	Finance



FULL OUTER JOIN

- FULL OUTER JOIN returns all rows when there is a match in either the left or right table.
- If there is no match, the result will include NULL for the missing side of the table. Essentially, it combines the results of both LEFT JOIN and RIGHT JOIN.

- **SELECT Employees.Name,
Employees.DepartmentID,
Departments.DepartmentName
FROM Employees
FULL JOIN Departments
ON Employees.DepartmentID =
Departments.DepartmentID;**

Name	DepartmentID	DepartmentName
John	101	HR
Sarah	102	IT
Michael	NULL	NULL
Emma	103	Marketing
NULL	NULL	Finance



Outer Join Example Tables

- *course* relation

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

- *prereq* relation

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

- Observe that

course information is missing CS-347

prereq information is missing CS-315



Left Outer Join

- General structure
 - *<table 1> natural left outer join <table 2>*
- Example
 - *course natural left outer join prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>

- Add to the *course* relation (on the left) the relevant info from *prereq*
 - Since CS-315 does not appear in *prereq* we use *null* for *prereq_id*
- In relational algebra syntax:
 $course \bowtie \text{prereq}$



Right Outer Join

- *course* natural right outer join *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

- Add to the *prereq* relation (on the right) the relevant info from *course*
- In relational algebra:

$course \bowtie r_{prereq}$



Full Outer Join

- *course* **natural full outer join** *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

- In relational algebra:
course ⋈ *prereq*



Consider the following three tables

- **salesperson**(ssn, name, start_year, dept_no)
- **trip**(ssn, from_city, to_city, depature_date,
□ return_date, trip_id)
- **expense**(trip_id, accountno, amount)



-
- 1) Find name, ssn of salesperson who went trip from Mangalore.
 - 2) Find name, year and dept of salesperson who went trip to agra.
 - 3) Find the amount of trip belonging to trip id=2
 - 4) Find the names of the salesperson who went to trip in year 2007.
 - 5) Find the names of the salesperson who started trip in 2008 and returned in 2009.
 - 6) Find the names, ssn of the salesperson who returned in 2009.
 - 7) Find the trip id of the salesperson who have departed in august 25.
 - 8) List all salesperson names with the cities they traveled to
 - 9) Find salespersons who have made more than one trip
 - 10) Find total expense of each salesperson
 - 11) Find salespersons who have taken at least one trip



1) Name, SSN of salesperson who went on trip from Mangalore

```
SELECT s.name, s.ssn  
FROM salesperson s  
JOIN trip t ON s.ssn = t.ssn  
WHERE t.from_city = 'Mangalore';
```

2) Name, start_year and dept of salesperson who went on trip to Agra

```
SELECT s.name, s.start_year, s.dept_no  
FROM salesperson s  
JOIN trip t ON s.ssn = t.ssn  
WHERE t.to_city = 'Agra';
```

3) Amount of trip belonging to trip_id = 2

```
SELECT SUM(amount) AS total_amount  
FROM expense  
WHERE trip_id = 2;
```



4) Names of salesperson who went on a trip in year 2007

```
SELECT DISTINCT s.name
FROM salesperson s
JOIN trip t ON s.ssn = t.ssn
WHERE t.depature_date BETWEEN '2007-01-01' AND '2007-12-31';
```

5) Names of salesperson who started trip in 2008 and returned in 2009

```
SELECT DISTINCT s.name
FROM salesperson s
JOIN trip t ON s.ssn = t.ssn
WHERE t.depature_date BETWEEN '2008-01-01' AND '2008-12-31'
AND t.return_date BETWEEN '2009-01-01' AND '2009-12-31';
```



7) Trip IDs where departure date is August 25 (any year)

```
SELECT trip_id  
FROM trip  
WHERE departure_date LIKE '____-08-25';
```

8) List all salesperson names with the cities they traveled to

```
SELECT s.name, t.to_city  
FROM salesperson s  
JOIN trip t ON s.ssn = t.ssn;
```

8) Find salespersons who have made more than one trip

```
SELECT s.name, COUNT(t.trip_id) AS trip_count  
FROM salesperson s  
JOIN trip t ON s.ssn = t.ssn  
GROUP BY s.name  
HAVING COUNT(t.trip_id) > 1;
```



10) Find total expense of each salesperson

```
SELECT s.name, SUM(e.amount) AS total_expense
FROM salesperson s
JOIN trip t ON s.ssn = t.ssn
JOIN expense e ON t.trip_id = e.trip_id
GROUP BY s.name;
```

11) Find salespersons who have taken at least one trip

```
SELECT name
FROM salesperson s
WHERE EXISTS (
    SELECT 1
    FROM trip t
    WHERE t.ssn = s.ssn
);
```




4) Names of salesperson who went on a trip in year 2007

```
SELECT DISTINCT s.name  
FROM salesperson s  
JOIN trip t ON s.ssn = t.ssn  
WHERE t.depature_date BETWEEN '2007-01-01' AND '2007-12-31';
```

5) Names of salesperson who started trip in 2008 and returned in 2009

```
SELECT DISTINCT s.name  
FROM salesperson s  
JOIN trip t ON s.ssn = t.ssn  
WHERE t.depature_date BETWEEN '2008-01-01' AND '2008-12-31'  
AND t.return_date BETWEEN '2009-01-01' AND '2009-12-31';
```



View Definition

- A **view** is a **virtual table** created using a query.
- It **does not store data physically**; instead, it stores the **query definition**.
- A view is defined using the **create view** statement which has the form

create view v as < query expression >

where <query expression> is any legal SQL expression. The view name is represented by v.

- **Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.**
- View definition is not the same as creating a new relation by evaluating the query expression
 - Rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.



View Definition and Use

- Create a view of instructors without their salary

create view *faculty* as

```
select ID, name, dept_name  
from instructor
```

- Find the names all instructors in the Biology department

```
select name  
from faculty  
where dept_name = 'Biology'
```

- Create a view of department salary totals

create view *departments_total_salary* as

```
select dept_name, sum (salary) as total_salary  
from instructor  
group by dept_name;
```



Views Defined Using Other Views

- One view may be used in the expression defining another view
- A view relation v_1 is said to **depend directly** on a view relation v_2 if v_2 is used in the expression defining v_1
- A view relation v_1 is said to **depend on** view relation v_2 if either v_1 depends directly to v_2 or there is a path of dependencies from v_1 to v_2
- A view relation v is said to be **recursive** if it depends on itself.



Views Defined Using Other Views Example

- Direct Dependency

```
CREATE VIEW biology_salary AS  
SELECT dept_name, total_salary  
FROM departments_total_salary  
WHERE dept_name = 'Biology';
```

biology_salary(v1) depends directly on
departments_total_salary (v2)



- **Indirect Dependency**

A view **v1** depends on **v2** if:

v1 depends directly on v2 OR

there is a chain of dependencies

biology_salary → departments_total_salary → instructor

- A view is **recursive** if it **depends on itself**.

CREATE VIEW **employee_hierarchy** AS

SELECT manager_id, employee_id

FROM **employee_hierarchy**;



Views Defined Using Other Views Example

- create view **physics_fall_2017** as
select **course.course_id**, **sec_id**, **building**,
room_number
from **course**, **section**
where *course.course_id = section.course_id*
and course.dept_name = 'Physics'
and section.semester = 'Fall'
and section.year = '2017';
- create view **physics_fall_2017_watson** as
select *course_id*, *room_number*
from **physics_fall_2017**
where *building = 'Watson';*



View Expansion (Cont.)

- **View Expansion** is the process where the DBMS replaces every view in a query with the SQL used to define that view, repeating this until only base tables remain.
- A way to define the meaning of views defined in terms of other views.
- Let view v_1 be defined by an expression e_1 that may itself contain uses of view relations.
- View expansion of an expression repeats the following replacement step:
 - repeat**
 - Find any view relation v_i in e_1**
 - Replace the view relation v_i by the expression defining v_i**
 - until no more view relations are present in e_1**
- **As long as the view definitions are not recursive, this loop will terminate**



View Expansion (Cont.)

- View expansion is the recursive replacement of view relations by their defining queries until only base relations remain, provided the views are non-recursive.
- ◆ **View 1**
 - **CREATE VIEW CS_Students AS**
 - **SELECT sid, name, marks**
 - **FROM Student**
 - **WHERE dept = 'CS';**
- ◆ **View 2**
 - **CREATE VIEW Top_CS AS**
 - **SELECT sid, name**
 - **FROM CS_Students**
 - **WHERE marks > 80;**



Expansion Process

- `SELECT * FROM Top_CS`

Step 1: Replace Top_CS

- `CREATE VIEW Top_CS AS`
- `SELECT sid, name`
- `FROM CS_Students`
- `WHERE marks > 80;`

Step 2: Replace CS_Students

`SELECT sid, name`

`FROM (`

`SELECT sid, name, marks`

`FROM Student`

`WHERE dept = 'CS'`

`)`

`WHERE marks > 80;`

Top_CS



CS_Students



Student table



View Expansion

- Expand the view :

```
create view physics_fall_2017_watson as  
  select course_id, room_number  
  from physics_fall_2017  
  where building= 'Watson'
```

- To:

```
create view physics_fall_2017_watson as  
  select course_id, room_number  
  from (select course.course_id, building, room_number  
        from course, section  
        where course.course_id = section.course_id  
              and course.dept_name = 'Physics'  
              and section.semester = 'Fall'  
              and section.year = '2017')  
  where building= 'Watson';
```

The database doesn't store data for a normal view. So it must **replace the view name with its query** before running.



Update of a View

- Add a new tuple to the *faculty* view which we defined earlier

insert into *faculty*

values ('30765', 'Green', 'Music');

Views don't store data So DB must insert into **instructor table**

- This insertion must be represented by the **insertion of the tuple into the *instructor* relation**
 - **Must have a value for salary**
- **Two approaches**
 - **Reject the insert ---standard practice**
 - **Insert the tuple with null**

('30765', 'Green', 'Music', null)

**INSERT INTO instructor
VALUES ('30765', 'Green',
'Music', NULL);**

create view *faculty* as into the *instructor* relation ---- if it allows null inserts

**select *ID*, *name*, *dept_name*
from *instructor***

Base table:

**instructor(*ID*, *name*,
dept_name, *salary*)**



Some Updates Cannot be Translated Uniquely

- create view *instructor_info* as
select *ID, name, building*
from *instructor, department*
where *instructor.dept_name = department.dept_name*;
- insert into *instructor_info*
values ('69987', 'White', 'Taylor');
- Issues??
 - Which table should this data go into?
 - Which department, if multiple departments in Taylor building?
 - What if no department or multiple dept is in Taylor building?
 - Which one should DB choose?
 - **THUS The INSERT is rejected.**

When a view is created using a JOIN, inserting through the view may require the DB to invent missing information. If it cannot uniquely determine base-table values, the update is not allowed.”

**instructor(ID, name, dept
name, salary)
department(dept name,
building, budget)**



And Some Cannot be Done at All

- **create view** *history_instructors* **as**
select *
from *instructor*
where *dept_name*= 'History';
- This view shows **only History department instructors** from the **instructor** table.
- What happens if we insert
('25566', 'Brown', 'Biology', 100000)
into *history_instructors*?

A view cannot be used to insert a row that violates the view's own WHERE condition.

Will this row appear in the view after insertion?

Insert is rejected outright

(DB enforces that inserted rows must appear in the view.)



View Updates in SQL

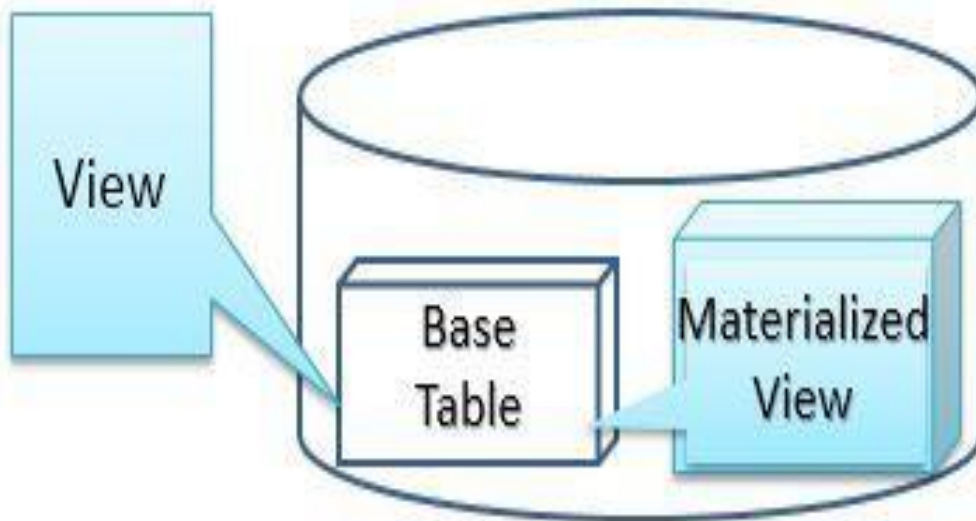
- Most SQL implementations allow updates only on simple views
 - The **from** clause has only **one database relation**.
 - The **select** clause contains only attribute names of the relation, and does not have any expressions, aggregates, or **distinct** specification.
 - `SELECT ID, salary*12` -- expression
 - `SELECT COUNT(*)` -- aggregate
 - `SELECT DISTINCT name`
 - Any attribute not listed in the **select** clause can be set to null
 - The query does not have a **group** by or **having** clause.



Materialized Views

- Certain database systems allow **view relations to be physically stored**.
 - **Physical copy created when the view is defined.**
 - **Such views are called **materialized view**:**
- If relations used in the query are updated, **the materialized view result becomes out of date**
 - Need to **maintain** the view, by updating the view whenever the underlying relations are updated.

REFRESH MATERIALIZED VIEW view_name;



```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name;
```

- **View updated after every change**
- **Updated only when refresh command runs**



Transactions

- A **transaction** is a **group of SQL operations** treated as **one unit of work**.
- A **transaction** consists of a **sequence of query and/or update statements** and is a “unit” of work
- The SQL standard specifies that a **transaction begins implicitly when an SQL statement is executed**.
- The transaction must end with one of the following statements:
 - **Commit work**. The updates performed by the transaction become permanent in the database. -Changes become **permanent (save)**
 - **Rollback work**. All the updates performed by the SQL statements in the transaction **are undone**. Database returns to old state(cancel everything)
- **Atomic transaction== All or Nothing**
 - **Either fully executed or rolled back as if it never occurred**
- Isolation from concurrent transactions
- One transaction's intermediate changes are hidden from others. Step 1: Deduct ₹1000
Step 2: Add ₹1000
- Transaction
= **Online shopping checkout(Add, order, pay)** If system crashes after Step 1:



Integrity Constraints

- **Integrity constraints** guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency.
 - A checking account must have a balance greater than \$10,000.00
 - A salary of a bank employee must be at least \$11.00 an hour
 - A customer must have a (non-null) **phone number**
 - **balance DECIMAL CHECK (balance > 10000)**
 - **salary DECIMAL CHECK (salary >= 11)**
 - **phone VARCHAR(15) NOT NULL**
- Database uses **Integrity Constraints** to enforce such rules
- Rules that ensure only **valid and consistent data** is stored.



Constraints on a Single Relation

- **not null**
- **primary key**
- **unique**
- **check (P)**, where P is a predicate



Not Null Constraints

- An attribute tagged with **not null** cannot have a null value
- Declare *name* and *budget* to be **not null**

name **varchar(20) not null**

budget **numeric(12,2) not null**



Unique Constraints

- **unique** ($A_1, A_2, \dots, A_m$)
 - The unique specification states that the attributes $A_1, A_2, \dots, A_m$ form a candidate key.
 - Candidate keys are permitted to be null (in contrast to primary keys).
 - SQL standard allows UNIQUE columns to contain NULL
 - No two rows can have the same combination of these attributes.
 - **UNIQUE(email)**
 - Then two customers **cannot share the same email**.



The check clause

- The **check (P)** clause **specifies a predicate P that must be satisfied by every tuple in a relation.**
- Database must restrict allowed values. That's done using CHECK constraints. This is a **Domain Constraint** because it restricts values of a column.
- Example: ensure that the value of semester is one of {fall, winter, spring or summer}

```
create table section
(course_id varchar (8),
 sec_id varchar (8),
 semester varchar (6),
 year numeric (4,0),
 building varchar (15),
 room_number varchar (7),
 time slot id varchar (4),
primary key (course_id, sec_id, semester, year),
check (semester in ('Fall', 'Winter', 'Spring', 'Summer')))
```

Only these 4 values allowed.



Referential Integrity

- Ensures that a value that appears in **one relation** for a given set of attributes **also appears for a certain set of attributes in another relation**.
 - Example: If “**Biology**” is a department name appearing in one of the tuples in the *instructor* relation, then there exists a tuple in the *department* relation for “**Biology**”.
- Let A be a set of attributes. Let R and S be two relations that contain attributes A and where A is the primary key of S.
- A is said to be a **foreign key** of R if for any values of A appearing in R these values also appear in S.
 - S is the referencing relation== table that has the primary key
 - R is the referenced relation= table that refers to it

Can an instructor belong to a department that doesn't exist?



Referential Integrity in SQL

- Database enforces this using a **FOREIGN KEY**
- A foreign key is an attribute in one relation whose values must match primary key values in another relation, ensuring referential integrity.
- Foreign *keys can be* specified as part of the SQL **create table** statement

foreign key (*dept_name*) **references** *department*

- By default, a foreign key references the primary-key attributes of the referenced table.
- SQL allows a list of attributes of the referenced relation to be specified explicitly.

foreign key (*dept_name*) **references** *department*
(*dept_name*)



Consider the following three tables

Schema Definition

- **Suppliers** (*sid*,, sname, address)
- **Parts** (*pid*, pname, color)
- **Catalog** (*sid*,*pid*, cost)



1. Find names of suppliers who supply some red part
2. Find the cheapest price for each part, including part name
3. Find sids of suppliers who charge more than the average cost for at least two parts
4. Find colors of parts that are supplied by fewer than 3 suppliers
5. Find names of suppliers who supply 'Part_X' (using IN)
6. Find suppliers who charge more for a part than the average cost of that part
7. Find the sids of suppliers who supply some red parts or at 221 packer street
8. Find the sids of suppliers who supply some red parts and some green parts
9. Find the pids of parts supplied by at least two different suppliers
10. For each color, find the number of parts where the average cost is at least 40



1) Find names of suppliers who supply some red part

- **SELECT DISTINCT s.sname**
- **FROM Suppliers s**
- **JOIN Catalog c ON s.sid = c.sid**
- **JOIN Parts p ON c.pid = p.pid**
- **WHERE p.color = 'red';**

2) Find the cheapest price for each part, including part name

- **SELECT p.pname, MIN(c.cost) AS cheapest_price**
- **FROM Parts p**
- **JOIN Catalog c ON p.pid = c.pid**
- **GROUP BY p.pname;**



3) Find sids of suppliers who charge more than the average cost for at least two parts

- **SELECT C.sid**
- **FROM Catalog C**
- **WHERE C.cost > (**
- **SELECT AVG(C1.cost)**
- **FROM Catalog C1**
- **WHERE C1.pid = C.pid)**
- **GROUP BY C.sid**
- **HAVING COUNT(DISTINCT C.pid) >= 2;**

4) Find colors of parts that are supplied by fewer than 3 suppliers

- **SELECT p.color**
- **FROM Parts p**
- **JOIN Catalog c ON p.pid = c.pid**
- **GROUP BY p.color**
- **HAVING COUNT(DISTINCT c.sid) < 3;**



5) Names of suppliers who supply **'Part_X'** (using IN)

- **SELECT sname**
- **FROM Suppliers**
- **WHERE sid IN (**
- **SELECT sid**
- **FROM Catalog**
- **WHERE pid IN (**
- **SELECT pid FROM Parts WHERE pname = 'Part_X'));**

6) Suppliers who charge more for a part than the average cost of that part

- **SELECT DISTINCT c.sid**
- **FROM Catalog c**
- **WHERE c.cost > (**
- **SELECT AVG(c2.cost)**
- **FROM Catalog c2**
- **WHERE c2.pid = c.pid);**



7) **sids of suppliers who supply some red parts OR at 221 packer street**

- **SELECT DISTINCT s.sid**
- **FROM Suppliers s**
- **WHERE s.address = '221 packer street'**
- **OR s.sid IN (**
- **SELECT c.sid**
- **FROM Catalog c**
- **JOIN Parts p ON c.pid = p.pid**
- **WHERE p.color = 'red');**

8) **sids of suppliers who supply some red AND some green parts**

- **SELECT sid**
- **FROM Catalog C, Parts P**
- **WHERE C.pid = P.pid AND P.color = 'red' mINTERSECT**
- **SELECT sid**
- **FROM Catalog C, Parts P**
- **WHERE C.pid = P.pid AND P.color = 'green'**



8) sids of suppliers who supply **some red AND some green parts**

- SELECT sid
- FROM Catalog c
- JOIN Parts p ON c.pid = p.pid
- WHERE p.color IN ('red','green')
- GROUP BY sid
- HAVING COUNT(DISTINCT p.color) = 2;

9) pids of parts supplied by **at least two suppliers**

SELECT pid

FROM Catalog

GROUP BY pid

HAVING COUNT(DISTINCT sid) >= 2;



10) For each color, find the number of parts where the average cost is at least 40:

- **SELECT p.color, COUNT(p.pid)**
- **FROM Parts p**
- **JOIN Catalog c ON p.pid = c.pid**
- **GROUP BY p.color**
- **HAVING AVG(c.cost) >= 40;**

OR

```
SELECT p.color, COUNT(*) AS num_parts  
FROM Parts p  
JOIN (  
    SELECT pid, AVG(cost) AS avg_cost  
    FROM Catalog  
    GROUP BY pid  
) c ON p.pid = c.pid  
WHERE c.avg_cost >= 40  
GROUP BY p.color;
```




Cascading Actions in Referential Integrity

- Referential integrity is a database constraint ensuring that relationships between tables remain consistent and valid. **If a value in one table refers to another table, that referred value must exist.** It requires that every foreign key value in a child table has a matching, existing primary key in the parent table,
- When a **referential-integrity constraint is violated**, the normal procedure is to **reject the action that caused the violation**. **Restrict/No Action:** Prevents the deletion or modification of a parent record if corresponding child records exist.

Department (Parent Table)

dept_name

Physics

CS

Course (Child Table)

course_id

C101

C102

dept_name

Physics

CS

INSERT INTO course VALUES ('C103', 'Biology');

DELETE FROM department WHERE dept_name = 'Physics';

- **Reject the action**
Rollback the transaction



Cascading Actions in Referential Integrity

- For foreign keys, in the case of a delete or update, instead of rejecting the action, **the system can take steps to change the tuple in the referencing relation to restore the constraint.**
- In the declaration of a foreign key, we can use:
 - **on delete cascade on update cascade,**
 - Automatically deletes or updates child records when the corresponding **parent record is changed or deleted.**
- For delete, If a delete of a tuple results in a referential-integrity constraint being violated, the system does not reject the delete. Instead, the delete cascades to the referencing relation, deleting the appropriate tuples.
- **Similarly for updates**



Cascading Actions Example

- Consider the declaration of *course*
create table *course* (
 (...
 dept_name **varchar**(20),
 foreign key (*dept_name*) **references** *department*
 on delete cascade
 on update cascade,
 ...)
- If we delete department Physics, then we also delete all courses offered in the Physics department
- Instead of cascade we can use :
 - **set null**-Keeps the child record but sets the foreign key field to empty (NULL).
 - **set default**



on update cascade:

- **UPDATE department**
- **SET dept_name = 'Physical Sciences'**
- **WHERE dept_name = 'Physics';**

course_id	dept_name
C101	Physical Sciences

SET Null

course_id	dept_name
C101	NULL

SET DEFAULT

If parent deleted → set to default value (like "Unknown")



Integrity Constraint Violation During Transactions

- Consider:

```
create table person (  
    ID char(10),  
    name char(40),  
    mother char(10),  
    father char(10),  
    primary key ID,  
    foreign key father references person,  
    foreign key mother references person)
```

Here a person references **other rows in the same table**.

- How to insert a tuple without **causing constraint violation**?
 1. **Insert father and mother of a person before inserting person**
 2. **OR, set father and mother to null initially, update after inserting all persons (not possible if father and mother attributes declared to be not null)**
 3. **OR defer constraint checking**



ID	name	father	mother
P3	Ravi	P1	P2

```
INSERT INTO person VALUES ('P3','Ravi','P1','P2');
```

But P1 and P2 are not inserted yet

Insert parents first

```
INSERT INTO person VALUES ('P1','Father',NULL,NULL);
```

```
INSERT INTO person VALUES ('P2','Mother',NULL,NULL);
```

```
INSERT INTO person VALUES ('P3','Ravi','P1','P2');
```

Insert with NULL, update later(Only works if father/mother allow NULL)

```
INSERT INTO person VALUES ('P3','Ravi',NULL,NULL);
```

```
UPDATE person SET father='P1', mother='P2' WHERE ID='P3';
```

Defer Constraint Checking

Check constraints at COMMIT time, not immediately.

Deferred constraint : The database waits until COMMIT to check foreign keys, so temporary rule-breaking is allowed — but the final result must be correct



Example: Deferred constraint

- create table person (
 - ID char(10) primary key,
 - name char(40),
 - mother char(10),
 - father char(10),
 - foreign key (father) references person **DEFERRABLE**,
 - foreign key (mother) references person **DEFERRABLE**
 -);
- BEGIN;
- **INSERT INTO person VALUES ('P3','Ravi','P1','P2');** -- parents not yet present
- **INSERT INTO person VALUES ('P1','Father',NULL,NULL);**
- **INSERT INTO person VALUES ('P2','Mother',NULL,NULL);**
- COMMIT;



Complex Check Conditions

- CHECK with Subquery

The predicate in the check clause can be an **arbitrary predicate that can include a subquery**.

check (*time_slot_id* in (select *time_slot_id* from *time_slot*))

Every *time_slot_id* used in **section** must exist in **time_slot** table.

The check condition states that the **time_slot_id** in each tuple in the **section relation** is actually the identifier of a time slot in the **time_slot relation**.

- The condition has to be checked not only when a **tuple is inserted or modified in section**, but also when the **relation time_slot changes**



CHECK with Subquery

time_slot

time_slot_id

T1

T2

section

course_id

CS101

time_slot_id

T1

INSERT INTO section VALUES ('CS102', 'T5');

DELETE FROM time_slot WHERE time_slot_id = 'T1';

So the DB must ensure consistency across **both tables**.

Most DB systems **do NOT allow subqueries inside CHECK** because they involve other tables.

When a constraint involves 2 (or) more tables, the table constraint mechanism is sometimes hard and results may not come as expected.



Assertions

- An **assertion** is a predicate expressing a condition that we wish the database always to satisfy.
- In SQL, an assertion is a statement that ensures that certain conditions are always enforced in the database no matter what.
- The following constraints, can be expressed using assertions:
 - For each tuple in the *student* relation, **the value of the attribute *tot_cred* must equal the sum of credits of courses that the student has completed successfully.**
 - An instructor **cannot teach in two different classrooms in a semester in the same time slot**
 - An assertion to **enforce that an employee's salary is never greater than their manager's salary**
- An assertion in SQL takes the form:
create assertion <assertion-name> **check** (<predicate>);



Example

- There must NOT exist any student whose tot_cred is less than 0.

create assertion no_negative_credit

check (not exists (

select *

from student

where tot_cred < 0

));

An assertion is a global rule that the entire database must always satisfy

CHECK → Rule for one table row

ASSERTION → Rule for the whole database

- » UPDATE student
- » SET tot_cred = -5
- » WHERE ID = 'S1';



Check/ Assertion

```
create table student (  
  ID int,  
  name varchar(20),  
  tot_cred int,  
  check (tot_cred >= 0)  
);
```

- In the **student table**, tot_cred cannot be negative

An **assertion** is not attached to one table. It checks a **condition involving many tables or many rows**.

```
create assertion credit_match  
check (  
  not exists (  
    select *  
    from student s  
    where s.tot_cred <>  
      (select sum(c.credits)  
       from takes t, course c  
       where t.course_id = c.course_id  
        and t.ID = s.ID)  ) );
```

For **every student**, tot_cred must equal sum of course credits



Example

student

ID	name	tot_cred
S1	Ravi	9
S2	Anu	4

takes

ID	course_id	grade
S1	CS101	A
S1	CS102	B
S2	CS101	A

course

course_id	credits
CS101	4
CS102	5

There must NOT exist any student whose stored credits are wrong.

`select * from student s`

s.tot_cred <> (sum of course credits)

Is student's stored total credit **different** from actual total?

**select sum(c.credits)
from takes t, course c
where t.course_id =
c.course_id
and t.ID = s.ID**



Built-in Data Types in SQL

- **date:** Dates, containing a (4 digit) **year, month and day**
 - Example: **date '2005-7-27'**
- **time:** Time of day, in **hours, minutes and seconds**.
 - Example: **time '09:00:30'**
 - Example: **time '09:00:30.75'**
- **timestamp:** **date plus time of day**
 - Example: **timestamp '2005-7-27 09:00:30.75'**
- **interval:** **period of time**
 - Example: **interval '1' day**
 - Subtracting a date/time/timestamp value from another gives an interval value
 - Interval values can be added to date/time/timestamp values
 - **SELECT DATE '2025-02-05' - DATE '2025-02-01'; →INTERVAL '4' DAY**
 - **SELECT DATE '2025-02-05' + INTERVAL '7' DAY;→ 2025-02-12**



User-Defined Types

- **create type** construct in SQL creates user-defined type

create type *Dollars* as numeric (12,2) final

- Example:

```
create table department  
(dept_name varchar (20),  
building varchar (15),  
budget Dollars);
```

Instead of writing `numeric(12,2)` everywhere, define **Dollars** once.



Domains

- A **domain** is like a type **plus rules**.
- **create domain** construct in SQL-92 creates user-defined domain types

create domain *person_name* char(20) not null

Now every column using this domain: Must be 20 characters and Cannot be NULL

- Types and domains are similar. **Domains can have constraints, such as not null, specified on them.**
- Example:

```
create domain degree_level varchar(10)  
constraint degree_level_test  
check (value in ('Bachelors', 'Masters',  
'Doctorate'));
```

```
create table student (  
    name varchar(20),  
    degree  
    degree_level  
);
```




Large-Object Types

- Large objects (photos, videos, CAD files, word files, etc.) are stored as a *large object*:
 - **blob**: binary large object -- object is a large collection of uninterpreted binary data (whose interpretation is left to an application outside of the database system).
 - **Photo of a student**
 - **Video of a lecture**
 - **clob**: character large object -- object is a large collection of character data
 - **Essay of a student**
- Large objects are usually **not stored in the tables**
- When a query returns a large object, **a pointer is returned rather than the large object itself**. Application uses pointer to fetch the file
- What can you do with large objects?
 - Read parts of a large object
 - **Update or replace**
 - Delete large objects

Student table

```
create table student (  
    ID int,  
    name varchar(20),  
    photo BLOB,  
    project_report CLOB  
);
```



Index Creation

- Many queries reference only a small proportion of the records in a table.
- It is inefficient for the system to read every record to find a record with particular value
- An index is a **special data structure** that helps the database find rows quickly.
- An **index on an attribute of a relation is a data structure** that allows the database system to find **those tuples in the relation that have a specified value for that attribute efficiently**, without scanning through all the tuples of the relation.
- We create an index with the **create index** command

create index <name> **on** <relation-name> (attribute);



Index Creation Example

- **create table *student***
(*ID* **varchar** (5),
name **varchar** (20) **not null**,
dept_name **varchar** (20),
tot_cred **numeric** (3,0) **default** 0,
primary key (*ID*))

- **create index *studentID_index* on *student*(*ID*)**

student	
ID	name
101	Ravi
102	Anu
103	Kiran
104	Meena

- Now database makes a small lookup table which **stores indexed value and Pointer to Row.**
- The query:

SELECT * FROM student WHERE ID = 103;

can be executed by using the index to find the required record, without looking at all records of *student*



Database Schema

Lives(Person_Name, Street, City)

Works(Person_Name, Company_Name, Salary)

Located(Company_Name, City)

Manager(Person_Name, Manager_Name)



Write Queries

1. Find the names of all employees who work for *First Bank Corporation*
2. Employees working in a company located in the same city they live
3. Employees who work for companies located in Bangalore
4. Average salary of employees in each company
5. Number of employees in each company
6. Companies with more than 5 employees
7. Employees earning more than the average salary
8. Employees who work in the same company as 'Ravi'
9. Employees who earn more than any employee in TCS
10. Managers who earn more than the employee they manage
11. Employees who do not work in companies located in their city
12. Employees earning the highest salary in each company
13. Employees who live in a city where no company is located
14. Employees who have no manager
15. Employees who work for a company located in **more than one city**



1. Find the names of all employees who work for *First Bank Corporation*

- SELECT Person_Name
- FROM Works
- WHERE Company_Name = 'First Bank Corporation';

Filters employees directly from the Works table based on company name.

2. Employees working in a company located in the same city they live

- SELECT W.Person_Name
- FROM Works W
- JOIN Lives L ON W.Person_Name = L.Person_Name
- JOIN Located C ON W.Company_Name = C.Company_Name
- **WHERE L.City = C.City;**

Employee's city (Lives) is compared with company's city (Located).



3. Employees who work for companies located in Bangalore

- SELECT W.Person_Name
- FROM Works W
- JOIN Located L ON W.**Company_Name** = L.**Company_Name**
- WHERE L.City = 'Bangalore';

Filters **companies located in Bangalore**, then finds employees working there.

4. Average salary of employees in each company

- SELECT **Company_Name**, **AVG(Salary) AS Avg_Salary**
- FROM Works
- GROUP BY **Company_Name;**

Groups employees **by company** and **computes average salary**.



5. Number of employees in each company

- SELECT Company_Name, COUNT(Person_Name) AS Employee_Count
- FROM Works
- GROUP BY Company_Name;

Counts employees per company.

6. Companies with more than 5 employees

- SELECT Company_Name
- FROM Works
- GROUP BY Company_Name
- HAVING COUNT(Person_Name) > 5;

HAVING is used to filter **groups**, not individual rows.



7. Employees earning more than the average salary

- SELECT Person_Name
- FROM Works
- WHERE Salary >
- (SELECT AVG(Salary) FROM Works);

Compares each employee's salary with the overall average.

8. Employees who work in the same company as 'Ravi'

- SELECT Person_Name
- FROM Works
- WHERE Company_Name = (
 - **SELECT Company_Name**
 - **FROM Works**
 - **WHERE Person_Name = 'Ravi');**

Inner query finds Ravi's company; outer query finds all employees in that company.



9 Employees who earn more than any employee in TCS

- **SELECT Person_Name**
- **FROM Works**
- **WHERE Salary > ANY (**
- **SELECT Salary FROM Works WHERE Company_Name = 'TCS'**
- **);**
- Salary is greater than **at least one** salary in TCS
 - **SELECT Person_Name**
 - **FROM Works**
 - **WHERE Salary >**
 - **(SELECT MIN(Salary)**
 - **FROM Works**
 - **WHERE Company_Name = 'TCS');**

■

If someone earns more than **the minimum salary in TCS**, then they earn more than **at least one** TCS employee.



10. Managers who earn more than the employee they manage

- SELECT M.Manager_Name
 - FROM Manager M
 - JOIN Works E ON M.Person_Name = E.Person_Name
 - JOIN Works MGR ON M.Manager_Name = MGR.Person_Name
 - WHERE MGR.Salary > E.Salary;
-
- Joins Works twice — once for employee salary and once for manager salary.



- Employees working in a company located in the same city they live or
- Employees who live in a city where their company is located
- (Using **EXISTS**)
 - SELECT W.Person_Name
 - FROM Works W
 - WHERE EXISTS (
 - **SELECT ***
 - **FROM Lives L, Located C**
 - **WHERE L.Person_Name = W.Person_Name**
 - **AND C.Company_Name = W.Company_Name**
 - **AND L.City = C.City**
 -)
- Checks whether at least one matching city exists for employee and company.



11. Employees who do not work in companies located in their city

```
SELECT W.Person_Name
FROM Works W
WHERE NOT EXISTS (
    SELECT *
    FROM Lives L, Located C
    WHERE L.Person_Name = W.Person_Name
        AND C.Company_Name = W.Company_Name
        AND L.City = C.City
);
```

- Selects employees for whom no matching city condition exists.



12. Employees earning the **highest salary** in each company

```
SELECT Person_Name, Company_Name
FROM Works W1
WHERE Salary = (
    SELECT MAX(Salary)
    FROM Works W2
    WHERE W1.Company_Name = W2.Company_Name
);
```

13. Employees who live in a city where no company is located

- SELECT L.Person_Name
- FROM Lives L
- WHERE L.City NOT IN (
 - **SELECT City**
 - **FROM Located**
-);



■ 14. Employees who have no manager

- SELECT Person_Name
- FROM Works
- WHERE Person_Name NOT IN (
 - SELECT Person_Name
 - FROM Manager
-)

15. Employees who work for a company located in more than one city

- SELECT W.Person_Name
- FROM Works W
- WHERE W.Company_Name IN (
 - **SELECT Company_Name**
 - **FROM Located**
 - **GROUP BY Company_Name**
 - **HAVING COUNT(DISTINCT City) > 1**
-);



Consider the following three tables

SAILORS(salid,sname,rating,age)

BOATS(bid,bname,color)

RESERVES(salid,bid,day)



1. Find the name of the sailors who have reserved the boat 103
2. Find the name of the sailors who have reserved at least one boat
3. Find the name of the sailors who have reserved a red boat
4. Find the names of the sailors who have reserved a red or a green boat
5. **Sailors who have NOT reserved any boat**
6. List **all boats**, including boats **that have never been reserved**, along with the sailor names.
7. Find the names and ages of all sailors
8. Find the sailors with a rating above 7.
9. Find the average age of all sailors
10. Count the total no of sailors.



1. Find the name of the sailors who have reserved the boat 103

```
SELECT DISTINCT S.sname  
FROM SAILORS S  
JOIN RESERVES R ON S.solid = R.solid  
WHERE R.bid = 103;
```

or

```
SELECT sname  
FROM SAILORS  
WHERE solid IN (  
    SELECT solid  
    FROM RESERVES  
    WHERE bid = 103 );
```



2. Find the name of the sailors who have reserved at least one boat

```
SELECT sname  
FROM SAILORS  
WHERE salid IN (  
    SELECT DISTINCT salid  
    FROM RESERVES );
```

Or

```
SELECT DISTINCT S.sname  
FROM SAILORS S  
JOIN RESERVES R ON S.salid = R.salid;
```



3) Sailors who have reserved a red boat

```
SELECT DISTINCT S.sname  
FROM SAILORS S  
JOIN RESERVES R ON S.solid = R.solid  
JOIN BOATS B ON R.bid = B.bid  
WHERE B.color = 'red';
```

OR

```
SELECT sname  
FROM SAILORS  
WHERE solid IN (  
    SELECT solid  
    FROM RESERVES  
    WHERE bid IN (  
        SELECT bid  
        FROM BOATS  
        WHERE color = 'red'  
    )  
)
```



4. Sailors who have reserved a **red OR green boat**

```
SELECT DISTINCT S.sname
FROM SAILORS S
JOIN RESERVES R ON S.solid = R.solid
JOIN BOATS B ON R.bid = B.bid
WHERE B.color IN ('red', 'green');
```

```
OR
SELECT sname
FROM SAILORS
WHERE solid IN (
    SELECT solid
    FROM RESERVES
    WHERE bid IN (
        SELECT bid
        FROM BOATS
        WHERE color IN ('red', 'green')    ));
```



5. Sailors who have NOT reserved any boat

```
SELECT S.sname
FROM SAILORS S
LEFT JOIN RESERVES R ON S.salid =
R.salid
WHERE R.salid IS NULL;
```



6. List **all boats**, including boats **that have never been reserved**, along with the sailor names.

```
SELECT S.sname, B.bname
FROM SAILORS S
RIGHT OUTER JOIN RESERVES R
    ON S.said = R.said
RIGHT OUTER JOIN BOATS B
    ON R.bid = B.bid;
```



7. Names and ages of all sailors

```
SELECT sname, age  
FROM SAILORS;
```

8. Sailors with a rating above 7

```
Select sid, sname ,rating,age  
.From sailors  
.Where rating>7;
```

9. Average age of all sailors

```
Select avg(age)  
from sailors;
```

10. Total number of sailors

```
Select count(*) from sailors;
```